

数据库MYSQL

欢迎来到数据库MYSQL学习资料分享！此文档是个人在学习MYSQL中积累的一些资料和个人总结，希望对看到的人有所帮助，同时也希望有知识、有能力的人一起完善这份文档

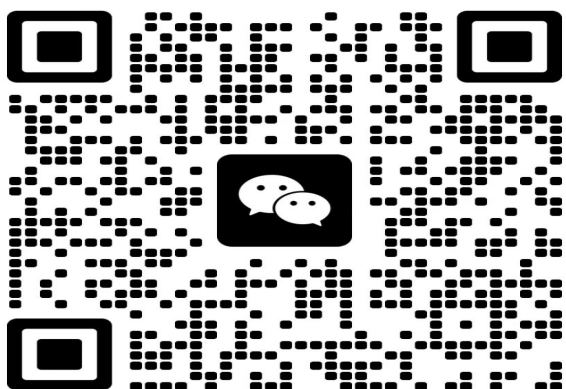
资料分享

面试八股总结



cpp辅导的阿甘

中国大陆 上海



扫一扫上面的二维码图案，加我为朋友。

本人微信感兴趣的可以加一下，
有什么学习或者求职上的疑惑都可以私信我，
尽量都会给大家解答的，并且周五晚上日常答疑
聊天也会给大家解答



cpp辅导的阿甘 向你推荐

奔跑中的cpp / c++



微信扫码加入星球

公众号 · cpp辅导的阿甘

这是自己创建的知识星球感兴趣的可以看看，星

球介绍可以看下面的链接：
<https://mp.weixin.qq.com/s/INb6nA5ci7GhJMrUTSBSjw>

资料分享

希望看到的人，能够对他的学习、工作有帮助，同时也希望有能力的人可以一起来完善此文档，让它可以发挥出更大的价值。

书籍分享

1. MySQL是怎样运行的

2. MySQL必知会

 [518_MySQL必知必会.pdf](#)

3. MySQL实战45讲

 [MySQL实战45讲.7z](#)

4. mysql基本语句学习（一下午基本可以掌握）

 [新建文件夹.7z](#)

 [MySQL培训日志.pdf](#)

 [bjpowernode.sql](#)

5. C++连接操作mysql学习（看下ppt即可）

 [c++连接数据库.7z](#)

6. T-SQL编程入门经典

 [T-SQL编程入门经典\(SQL Server 2008\).pdf](#)

网站分享

面试八股总结

希望看到的人，能够对他的学习、工作有帮助，同时也希望有能力的人可以一起来完善此文档，让它可以发挥出更大的价值。

MySQL执行流程

- 连接器
- 查询缓存
- 解析器（解析SQL）
- 执行SQL（预处理器、优化器、执行器）

MySQL存储引擎

引擎的类型

InnoDB、MyISAM、Memory、Merge

各类引擎对比

- InnoDB具备外键支持功能的事务处理引擎
- MyISAM主要的非事务处理存储引擎
- Memory置于内存的表
- Merge用来管理由多个MyISAM表构成的表集合

MySQL索引

概念

MySQL索引是一种数据结构，用于加快数据库表中数据的检索速度。类似于书籍的索引，可以帮助数据库系统快速定位和访问存储在表中的数据。索引通过创建一个按照特定列或列组合排序的数据结构，可以有效地减少数据库系统需要扫描的数据量，从而提高查询性能。（可以再讲下索引的数据结构，就是下面的（索引原理（为什么是B+树）））

索引的类型介绍

- 按物理存储结构分类：聚簇索引（主键索引）、二级索引（辅助索引）
- 按字段特性分类：主键索引、唯一索引、普通索引、前缀索引
- 按字段个数分类：单列索引、联合索引

什么时候需要创建索引

- 字段有唯一性限制的，比如商品编码
- 经常用于where查询条件的字段，这样能够提高整个表的查询速度，如果查询条件不是一个字段，可以建立联合索引。
- 经常用于group by和order by的字段，这样在查询的时候就不需要再去做一次排序了，因为我们都已经知道了建立索引之后在B+Tree中的记录都是排序好的。

什么时候不需要创建索引

- where条件，group by，order by里用不到的字段，索引的价值是快速定位，如果起不到定位的字段通常是不需要创建索引的，因为索引是会占用物理空间的
- 字段中存在大量重复数据，不需要创建索引，比如性别字段，只有男女，如果数据库表中，男女的记录分布均匀，那么无论搜索哪个值都可能得到一半的数据。在这些情况下，还不如不要索引，因为Mysql还有一个查询优化器，查询优化器发现某个值出现在表的数据行中的百分比很高的时候，它一般会忽略索引，进行全表扫描
- 表数据太少的时候，不需要创建索引
- 经常更新的字段不用创建索引，比如不要对电商项目的用户余额建立索引，因为索引字段频繁修改，由于要维护B+Tree的有序性，那么就需要频繁的重建索引，这个过程会影响数据库性能的

索引原理（为什么是B+树）

- MySQL的数据是持久化的，并且数据（索引+记录）是保存在磁盘上的。因此设计选用的数据结构的时候不仅要考虑检索的时间效率，还要考虑磁盘I/O的操作次数。并且MySQL是支持范围查找的，所以索引的数据结构不仅要能高效地查询某一个记录，而且也要能高效地执行范围查找。
- 如果用数组来存储的话，可以使用二分查找，时间复杂度为 $O(\log n)$ ，效率较高。但是插入新元素的话，就会涉及到数据的移动之类的，效率较低。这时想到了可采用一种既可以适合二分查找的，用的是非线性的存储结构
- 这时想到了二叉搜索树（一个节点的左子树的所有节点都小于这个节点，右子树的所有节点都大于这个节点）。二叉查找树是一个跳跃结构，不必连续排列，这样在插入的时候，新节点可以放在任何位置，不会像线性结构那样插入一个元素，所有元素都需要向后排序。但是采用这种结构也是存在问题的，如果当每次插入的元素都是二叉搜索树中最大的元素，那二叉搜索树就会退化成一条链表，查找数据的时间复杂度变成了 $O(n)$ 。并且由于树是存储到磁盘中的，树的高度就等于每次查询数据时磁盘IO操作的次数，所以树的高度越高，就会影响查询性能。
- 为了解决二叉查找树退化成链表的极端情况，便想到了平衡二叉查找树（AVL树）。平衡二叉搜索树（每个节点的左子树和右子树的高度差不能超过1）这样查询操作的时间复杂度就会一直维持在 $O(\log n)$ 。但是仍未解决访问磁盘次数的问题，会随着插入的元素增多，而导致树的高度变高，这意味着磁盘I/O操作次数多，会影响整体数据查询的效率。
- 这是想到了用多叉树B树，降低层高。B树（每一个节点最多可以包括M个子节点，M称为B树的阶，所以B树就是一个多叉树。）。B树的。但是在查询位于底层的某个节点（比如A记录）过程中，非A记录节点里的记录数据会从磁盘加载到内存，但是这些记录数据是没用的，我们只是想读取这些节点的索引数据来做比较查询，而非A记录节点
- [一文彻底搞懂MySQL基础：B树和B+树的区别_mysql b树和b+树的区别-CSDN博客](#)
- B+树 VS Hash
 - Hash在做等值查询的时候效率贼快，搜索复杂度为 $O(1)$ 。但是Hash表不适合做范围查询，它更适合做等值查询，这也是B+Tree索引要比Hash表索引有着更广泛的使用场景的原因

防止索引失效的方法

- 对索引使用左或者左右模糊匹配：（索引 B+ 树是按照「索引值」有序排列存储的，只能根据前缀进行比较。）
- 对索引使用函数（因为索引保存的是索引字段的原始值，而不是经过函数计算后的值，自然就没办法走索引了。）从 MySQL 8.0 开始，索引特性增加了函数索引，即可以针对函数

计算后的值建立一个索引，也就是说该索引的值是函数计算后的值，所以就可以通过扫描索引来查询数据。

- 对索引进行表达式计算，索引保存的是索引字段的原始值，而不是 $id + 1$ 表达式计算后的值，所以无法走索引，**只能通过把索引字段的取值都取出来**，然后依次进行表达式的计算来进行条件判断，因此采用的就是全表扫描的方式。
- 对**索引隐式类型转换**，如果索引字段是字符串类型，但是在条件查询中，输入的参数是整型的话，你会在执行计划的结果发现这条语句会走全表扫描。走不走索引还是要看转换规则的，MySQL 在遇到字符串和数字比较的时候，**会自动把字符串转为数字，然后再进行比较**。
- 联合索引非最左匹配联合索引要能正确使用需要遵循最左匹配原则，也就是按照最左优先的方式进行索引的匹配。
- where子句中的 OR。在 WHERE 子句中，如果在 OR 前的条件列是索引列，而在 OR 后的条件列不是索引列，那么索引会失效。（**除非两个都是索引列**）

索引的优化

- 前缀索引：减小索引字段的大小，增加一个索引页中存储的索引值，有效提高索引的查询速度
- 覆盖索引：从二级索引中查询得到记录，而不需要通过聚簇索引查询获得，可以避免回表的操作
- 主键索引最好是自增的：插入一条新记录，都是追加操作，不需要重新移动数据，因此这种插入数据的方法效率非常高。如果我们使用非自增主键，那么插入数据的时候可能还会设计到页分裂，而页分裂还有可能会造成大量的内存碎片，导致索引结构不紧凑，从而影响查询效率。
- 主键字段的长度不要太大，因为主键字段长度越小，意味着二级索引的叶子节点越小（二级索引的叶子节点存放的数据是主键值），这样二级索引占用的空间也就越小
- 索引最好设置为 NOT NULL：为 NULL 的列会使索引、索引统计和值比较都更复杂，比如进行索引统计时，count 会省略值为 NULL 的行。NULL 值是一个没意义的值，但是它会占用物理空间

索引的选择

- 如果有主键，默认会使用主键作为聚簇索引的索引键（key）
- 如果没有主键，就选择第一个不包含NULL值的唯一列作为聚簇索引的索引键（key）
- 在上面两个都没有的情况下，InnoDB将自动生成一个隐式自增id列作为聚簇索引的索引键

(key)

使用like “%x”，索引一定会失效吗？

- 如果表里只有主键字段和辅助索引字段
 - 那么查询的数据都在二级索引的B+树，因为二级索引的B+树的叶子节点包含索引值+主键值，所以查二级索引的B+数就能查找到全部结果了（简称，覆盖索引）
 - 如果模糊查询使用的是左模糊，那么会通过全扫描二级索引的B+树的方式查询到数据
 - 其他的查询的话，会对索引列进行范围查询
- 如果表里还有其他字段
 - 进行向左模糊查询的时候，优化器会认为直接选择全表扫描的方式来查询数据

MySQL事务

概念

MySQL事务是一组数据库操作的执行单元，被视为一个不可分割的工作单元，要么全部成功执行，要么全部回滚到初始状态，以保持数据库的一致性和完整性。事务是用来管理对数据库的并发访问和保证数据的一致性的重要机制。（然后再把事务特性说一下）

事务特性（重点）

- 原子性：（通过undo log（回滚日志）来保证）
- 一致性：事务操作前和操作后，数据满足完整性约束，数据库保持一致性状态。（通过持久性+原子性+隔离性保证）
- 隔离性：数据库允许多个并发事务同时对其数据进行读写和修改的能力，隔离性可以防止多个事务并发执行时由于交叉 执行而导致数据的不一致，因为多个事务同时使用相同的数据时，不会相互干扰，每个事务都有一个完整的数据空间，对其他并发事务是隔离的。（通过MVCC（多版本并发控制）或锁机制来保证）
- 持久性：

事务并发引发的问题

- 脏读：如果一个事务读到了另一个**未提交事务修改过的数据**，就意味着发生了脏读现象。（如果未提交的事务发生了**回滚**，那么这个事务刚才读到的数据就是过期数据，这种现象

称为脏读)

- 不可重复读：在一个事务内多次读取同一个数据，如果出现前后两次读到的数据不一样的情况，就意味着发生了不可重复读现象。（比如有两个事务A和B在同时处理，事务A先读取了某条数据，在这过程中如果事务B更新了这条数据，并提交了事务，那么当事务A再次读取该数据时，就会发现前后两次读到的数据是不一致的，这种现象称为不可重复读）
- 幻读：在一个事务内多次查询某个符合查询条件的记录数量，如果出现前后两次查询到的记录数量不一样的情况，就意味着发生了幻读现象。

事务隔离级别

- 读未提交：指一个事务还没提交时，它做的变更就能被其他事务看到
- 读提交：指一个事务提交之后，它做的变更才能被其他事务看到
- 可重复读：指一个事务执行过程中看到的数据，一直跟这个事务启动时看到的数据是一致的（MySQL的InnoDB引擎默认的隔离级别）
- 串行化：会对记录加上读写锁，在多个事务对这条记录进行读写操作时，如果发生读写冲突的时候，后访问的事务必须等前一个事务执行完成时，才能继续执行

MVCC原理

- 在InnoDB存储引擎的数据库表中，它的聚簇索引记录中都包含两个隐藏列：
 - trx_id：当一个事务对某条聚簇索引记录进行改动时，就会把该事务的事务id记录在trx_id隐藏列里；
 - roll_pointer，每次对某条聚簇索引记录进行改动时，都会把旧版本的记录写入到undo日志中，然后这个隐藏列是个指针，指向每一个旧版本记录，于是就可以通过它找到修改前的记录。（会对这个更改记录生成一个undo_log版本链）

Read View在MVCC里的工作方式

- Read View是“快照读”SQL执行时MVCC提取数据的依据；快照读就是最普通的select查询SQL语句；当前读指待执行insert、update、delete、select... for update、select ... lock in share mode时进行数据读取的方式
- Read View其实就是一个数据结构，包含4个字段
 - m_ids：指的是在创建Read View时，当前数据库中活跃事务的事务id列表，注意是一

个列表，“活跃事务”指的就是，启动了但还没提交的事务。

- min_trx_id:指的是在创建Read View时，当前数据库中活跃事务中事务id最小的事务，也就是m_ids的最小值
 - max_trx_id: 这个并不是m_ids的最大值，而是创建Read View时当前数据库中应该给下一个事务的id值，也就是全局事务中最大的事务id值+1；
 - creator_trx_id:指的是创建该 Read View 的事务的事务 id。
- 聚簇索引的两个隐藏列
- trx_id: 当一个事务对某条聚簇索引记录进行改动时，就会把该事务的事务 id 记录在 trx_id 隐藏列里；
 - roll_pointer, 每次对某条聚簇索引记录进行改动时，都会把旧版本的记录写入到 undo 日志中，然后这个隐藏列是个指针，指向每一个旧版本记录，于是就可以通过它找到修改前的记录。

读提交工作方式

- 读提交隔离级别是在每次读取数据时，都会生成一个新的Read View
- 查询数据的时候，会依据版本链数据的访问规则：
 - 判断当前事务id是否等于creator_trx_id，成立说明数据就是自己这个事务更改的，可以访问
 - 判断trx_id是否小于min_trx_id，成立说明数据已经提交了，可以访问
 - 判断trx_id是否大于max_trx_id，成立说明该事务是在readView生成以后才开启，不允许访问
 - 判断trx_id是否大于等于min_trx_id并且小于等于max_trx_id,成立在m_ids数据列表中对比，不存在数据则代表数据是已提交的，可以访问

可重复读工作方式

- 可重复读隔离级别是启动事务时生成一个Read View，然后整个事务期间都在用这个Read View
- 连续多次快照读，readView会产生复用，没有幻读问题
- 两次快照读之间存在当前读，ReadView会重新生成，导致产生 幻读（）

可重复读隔离级别对幻读的解决

- 针对**快照读**（普通select语句），是**通过MVCC方式解决了幻读**，因为可重复读隔离级别下，事务执行过程中看到的数据，**一直跟这个事务启动时看到的数据是一致的**，即使中途有其他事务插入了一条数据，是查询不出来这条数据的，所以就很好避免了幻读问题。
- 针对**当前读**（select ... for update等语句），是**通过next-key lock（记录锁+间隙锁）方式解决了幻读**，因为当执行select ... for update语句的时候，会加上next-key lock，如果有其他事务在next-key lock锁范围内插入了一条记录，那么这个插入语句就会被阻塞，无法成功插入，所以就很好避免了幻读问题

可重复读怎么解决幻读

- InnoDB引擎为了解决可重复读隔离级别使用当前读而造成的幻读问题，就引出**间隙锁**
- 过程：
 - 当一个事务执行查询语句后，会在这个 范围内加上**间隙锁+记录锁**的组合
 - 当其他事务执行插入语句时，判断到插入位置被其他事务插入了**间隙锁+记录锁**，于是当前事务就会生成一个插入意向锁，同时进入等待状态，直到其他事务进行了提交，这样就避免了由于当前事务插入新记录而导致事务A发生了幻读的现象
- 可重复读级别虽然很大程度上避免了幻读，但是还是没有能完全解决幻读
 - 场景一：比如当前事务A第一次执行普通查询select语句时生成了一个ReadView，之后事务B向表中插入一条id的记录并提交。接着，事务A对这个id条记录进行更新操作，在这个时刻，这条新记录的trx_id隐藏列的值就变成了事务A的事务id，之后事务A再使用普通select语句去查询 这条记录时就可以看到这条记录了，于是就发生了幻读。
 - 场景二：先事务A 执行了一个快照读语句，然后事务B执行了一条插入语句；紧接着事务A又执行了一条当前读语句，就发生了幻读。要避免这类特殊场景下发生幻读的现象的话，就是尽量在开启事务之后，马上执行 select ... for update 这类当前读的语句，因为它会对记录加 next-key lock，从而避免其他事务插入一条新记录

MySQL锁

锁的种类

■ 全局锁

- 加锁之后，整个数据库就处于只读状态了
- 应用场景：
 - 全局锁主要用于全库逻辑备份
- 加全局锁又会带来什么缺点
 - 造成业务停滞
- 既然备份数据库数据的时候，使用全局锁会影响业务，那有什么其他方式可以避免？
 - 数据库的引擎支持的事务 **支持可重复读的隔离级别**，那么在备份数据库之前先开启事务，会先创建 Read View，然后整个事务执行期间都在用这个 Read View，而且由于 MVCC 的支持，备份期间业务依然可以对数据进行更新操作。

■ 表级锁

- 表锁
 - 表锁除了会限制别的线程的读写外，也会限制本线程接下来的读写操作
 - 不过尽量避免在使用 InnoDB 引擎的表使用表锁，因为表锁的颗粒度太大，会影响并发性能，**InnoDB 牛逼的地方在于实现了颗粒度更细的行级锁。**
- 元数据锁（MDL）
 - 当对数据库表进行操作时，会 **自动给这个表** 加上 MDL；
 - 对一张表进行 CRUD 操作时，加的是 MDL 读锁；（防止其他线程对这个表结构做变更）
 - 对一张表做结构变更操作的时候，加的是 MDL 写锁
 - MDL 不需要显示调用，那它是在什么时候释放的？
 - MDL 是在事务提交后才会释放，这意味着事务执行期间，MDL 是一直持有的；长连接容易造成阻塞
 - 为什么线程 C 因为申请不到 MDL 写锁，而导致后续的申请读锁的查询操作也会被阻塞？
 - 申请 MDL 锁的操作会形成一个队列，队列中写 **锁获取优先级高于读锁**，一旦出现 MDL 写锁等待，会阻塞后续该表的所有 CRUD 操作。
 - 所以为了能安全的对表结构进行变更，在对表结构变更前，先要看看数据库中的长事务，是否有事务已经对表加上了 MDL 读锁，如果可以考虑 kill 掉这个长事务，然后再做表结构的变更
- 意向锁
 - 在使用 InnoDB 引擎的表里对某些记录加上 **「共享锁」** 之前，需要先在表级别加上一个 **「意向共享锁」**

- 在使用 InnoDB 引擎的表里对某些纪录加上「独占锁」之前，需要先在表级别加上一个「意向独占锁」；
- 也就是，当执行插入、更新、删除操作，需要先对表加上「意向独占锁」，然后对该记录加独占锁。而普通的 select 是不会加行级锁的，普通的 select 语句是利用 MVCC 实现一致性读，是无锁的。
- 意向共享锁和意向独占锁是表级锁，不会和行级的共享锁和独占锁发生冲突，而且意向锁之间也不会发生冲突，只会和共享表锁 (*lock tables ... read) 和独占表锁 (lock tables ... write) 发生冲突。
- 表锁和行锁是满足读读共享、读写互斥、写写互斥的。

- auto——inc锁

- 在插入数据时，可以不指定主键的值，数据库会自动给主键赋值递增的值，这主要是通过auto-inc锁的
- auto-inc锁不再是一个事务提交后才释放，而是再执行完后插入语句后就会立即释放
- 在插入数据时，会加入一个表级别的auto-inc锁。那么一个事务在持有auto-inc锁的过程中，其他事务如果要向该表插入语句都会被阻塞，从而保证插入数据时，被auto_increment修饰的字段的值是连续递增的
- 在 MySQL 5.1.22 版本开始，InnoDB 存储引擎提供了一种轻量级的锁来实现自增。一样也是在插入数据的时候，会为被 AUTO_INCREMENT 修饰的字段加上轻量级锁，然后给该字段赋值一个自增的值，就把这个轻量级锁释放了，而不需要等待整个插入语句执行完后才释放锁

- 行级锁

- Record Lock (记录锁)

- 锁住的是一条记录。而且记录锁有S锁（共享锁）和X锁（独占锁）之分
- 共享锁（S锁）满足读读共享，读写互斥。独占锁（X锁）满足写写互斥、读写互斥。

- Gap Lock (间隙锁)

- 只存在于可重复读隔离级别，目的是为了解决可重复读级别下幻读的现象
- 间隙锁虽然存在 X 型间隙锁和 S 型间隙锁，但是并没有什么区别，间隙锁之间是兼容的，即两个事务可以同时持有包含共同间隙范围的间隙锁，并不存在互斥关系，因为间隙锁的目的是防止插入幻影记录而提出的

- Next-Key Lock (临键锁)

- 是记录锁和间隙锁的组合，锁定一个范围，并且锁定记录本身
- next-key lock 是包含间隙锁+记录锁的，如果一个事务获取了 X 型的 next-key

lock, 那么另外一个事务在获取相同范围的 X 型的 next-key lock 时, 是会被阻塞的

- 插入意向锁
 - 插入意向锁名字虽然有意向锁, 但是它并不是意向锁, 它是一种特殊的间隙锁, 属于行级别锁

阿甘(42379064)

阿甘(42379064)